

1. Perbedaan Top-level Container, Intermediate Container, dan Atomic Component:
 - Top-level Container: Wadah utama yang menjadi fondasi dasar dari sebuah jendela aplikasi dan berdiri sendiri (*standalone*), serta terhubung dengan sistem operasi (misalnya jendela window, *title bar*, tombol minimize/maximize/close).
 - Contoh: JFrame (atau JDialog).
 - Intermediate Container: Wadah perantara yang diletakkan di dalam top-level container untuk menampung, mengelompokkan, dan mengatur tata letak komponen-komponen lainnya.
 - Contoh: JPanel (atau JScrollPane).
 - Atomic Component (Leaf Component): Komponen dasar yang berdiri sendiri untuk berinteraksi langsung dengan pengguna atau menampilkan informasi spesifik, dan tidak dapat menampung komponen lain di dalamnya.
 - Contoh: JButton (atau JLabel, JTextField).
2. Mengapa kedua JRadioButton perlu diberi properti buttonGroup yang sama?
 - Agar JRadioButton tersebut bersifat eksklusif (saling mengunci), di mana pengguna hanya dapat memilih satu opsi saja dalam satu waktu. Jika properti buttonGroup tidak disatukan, semua tombol radio akan dapat dicentang secara bersamaan seperti kotak centang (*checkbox*).
3. Kapan Anda memilih JComboBox dibandingkan JRadioButton?
 - JComboBox dipilih ketika pilihan data yang tersedia jumlahnya sangat banyak atau dinamis (misalnya daftar kota, program studi, atau tahun) sehingga lebih hemat ruang tampilan karena disembunyikan dalam bentuk menu drop-down.
 - JRadioButton dipilih ketika pilihan opsinya sangat sedikit (umumnya antara 2 sampai 4 opsi) dan penting untuk ditampilkan secara langsung agar pengguna bisa melihat semua pilihan sekaligus tanpa harus mengklik menu terlebih dahulu.
4. Mengapa kode di dalam initComponents() tidak boleh diedit langsung, dan di mana kode tambahan seharusnya ditulis?
 - Kode di dalam initComponents() tidak boleh diedit secara manual karena bagian tersebut dikelola dan digenerate secara otomatis oleh *Form Editor* (Visual Designer) NetBeans. Jika diedit secara langsung, perubahan tersebut dapat merusak sinkronisasi tampilan grafis atau akan tertimpa otomatis saat Anda memodifikasi komponen melalui tab *Design*.
 - Kode tambahan (seperti logika program, event handler, atau method kustom tambahan) seharusnya ditulis di luar blok yang dilindungi tersebut, biasanya di bagian bawah class atau di dalam method/constructor setelah pemanggilan initComponents().
5. Mengapa FlatLightLaf.setup() harus dipanggil sebelum form dibuat?
 - Karena pengaturan *Look and Feel* (pustaka tema grafis seperti FlatLaf) harus diinisialisasi dan diterapkan ke sistem sebelum komponen antarmuka (*UI components*)

dibuat dan dimunculkan ke layar. Jika dipanggil setelah form dibuat, komponen sudah telanjur dirender menggunakan tema bawaan sistem (*default* Java Metal/Cross-Platform) sehingga tema modern FlatLaf tidak akan teraplikasikan dengan benar pada jendela aplikasi.